Software Requirements Specification

FinManage

Personal Finance Management System

Version 1.4

Capstone Project — MERN Stack Development

September 2026

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for FinManage, a full-stack personal finance management web application. It is intended for the development team, project evaluators, and reviewers, and serves as the authoritative reference for what the system must do, who will use it, and the constraints under which it will operate.

1.2 Scope

FinManage allows an individual user to track multiple financial accounts, record and categorize transactions, and view analytical reports of their financial activity over time. The platform supports multiple registered users, with each user's data kept private and isolated from all others. It is built on the MERN stack (MongoDB, Express.js, React, Node.js), secured with JWT-based authentication, and deployable to a cloud environment (AWS) using Docker.

The following are explicitly out of scope for this version:

- Integration with live bank feeds or payment gateways (e.g., Plaid, Razorpay)
- Real money transfers or actual payment processing
- Investment portfolio tracking (stocks, mutual funds, crypto)
- Tax computation or filing assistance

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification
JWT	JSON Web Token — used for stateless authentication
MERN	MongoDB, Express.js, React.js, Node.js
CRUD	Create, Read, Update, Delete
API	Application Programming Interface
MoSCoW	Prioritization method: Must have, Should have, Could have, Won't have

1.4 Document Overview

Section 2 describes the product at a high level, including user classes and constraints. Section 3 details functional requirements module by module. Section 4 covers non-functional requirements. Section 5 defines external interfaces. Section 6 specifies database requirements. Section 7 outlines system architecture, and Section 8 summarizes requirement prioritization.

2. Overall Description

2.1 Product Perspective

FinManage is a new, standalone, self-contained web application. It is not a modification of an existing system and has no dependency on external financial institutions. It follows a standard three-tier architecture: a React single-page frontend, an Express/Node REST API backend, and a MongoDB data store.

2.2 Product Feature Summary

- Secure user registration, login, and session management (JWT-based)
- Multi-account management (e.g., Cash, Bank, Credit Card, Savings), with balance = opening balance + income − expense
- Transaction logging with categorization and filtering

- Analytical dashboard: income vs. expense trends, category breakdowns
- CSV export of transaction and report data (stretch goal)

2.3 User Classes and Characteristics

User Role	Description	Technical Expertise
Registered User	The single functional role in v1.0. Owns one or more accounts, records transactions, and views personal reports. Full CRUD access limited strictly to their own data.	General public; no technical expertise assumed

Note: each registered user's data is private and fully isolated from other users — there is no shared household/joint account access and no separate administrative role. Both are deliberate product decisions: the application is designed as a focused, individual personal-finance tool, not a multi-role platform. They are candidates for a future version, not gaps in the current one.

2.4 Operating Environment

- Client: modern evergreen browsers (Chrome, Firefox, Edge) — desktop and mobile responsive
- Server: Node.js runtime (v18+), Express.js framework
- Database: MongoDB (Atlas cloud-hosted or self-managed)
- Deployment: Dockerized containers hosted on an AWS EC2 instance

2.5 Design and Implementation Constraints

- Must be built using the MERN stack only (no external ML/AI services or third-party delivery infrastructure)
- Authentication must use JWT-based security practice
- Must include automated test coverage (Jest / React Testing Library) for core business logic, notably balance aggregation and category-wise reporting

2.6 Assumptions and Dependencies

- Users have basic familiarity with personal finance terms (account, category, budget)
- The system hardcodes a single currency (INR) for all users in v1.0; there is no per-user currency field in the Users schema and no per-user currency setting. Multi-currency support is not guaranteed for this version.
- Internet connectivity is required; there is no offline mode
- MongoDB Atlas free-tier availability is assumed for development and demo purposes

3. System Features and Functional Requirements

Each module below lists its functional requirements (FR) with a unique identifier and a MoSCoW priority. A consolidated priority table appears in Section 8.

3.1 Module: User Authentication & Account Management

ID	Requirement	Priority
FR-1.1	The system shall allow a new user to register using name, email, and password.	Must
FR-1.2	The system shall validate email uniqueness and enforce a minimum password strength.	Must
FR-1.3	The system shall authenticate users via email/password and issue a JWT on success.	Must
FR-1.4	The system shall protect all authenticated routes using JWT verification middleware, with explicit exceptions for registration, login, and the password-reset request/confirm endpoints, which validate a single-use reset token instead of a JWT.	Must
FR-1.6	The system shall allow users to log out by deleting the JWT client-side. Server-side token revocation (blocklisting) is not implemented in this version; a logged-out token remains technically valid until its natural expiry.	Must
FR-1.7	The system shall support password reset via a hashed, time-limited token: on request, the system generates a token, stores its hash and expiry (resetPasswordToken, resetPasswordExpires) against the user, and the reset link/token is surfaced directly in the UI for this version rather than delivered by email. Real SMTP delivery (e.g., via Nodemailer) is a direct swap-in for production and does not change the underlying token logic.	Must

3.2 Module: Financial Account Management

ID	Requirement	Priority
FR-2.1	The system shall allow a user to create multiple financial accounts (e.g., Cash, Bank, Credit Card, Savings), each with a name, type, and opening balance.	Must
FR-2.2	The system shall compute each account's current balance on read, via a query aggregating its transactions, rather than maintaining a separately stored, incrementally-updated balance field: balance = openingBalance + Σ(income) − Σ(expense). This guarantees the balance is always correct relative to the transaction log, at the cost of recomputing it on each request.	Must
FR-2.3	The system shall allow a user to edit an account's name or type, and to delete an account only if no Transactions reference it (see Section 6.5). There is no archive/soft-delete state in this version.	Must

3.3 Module: Transaction Management

ID	Requirement	Priority
FR-3.1	The system shall allow a user to record a transaction with amount, type (income/expense), a reference to a Category (categoryId), account, date, and optional notes.	Must
FR-3.2	The system shall allow a user to edit or delete an existing transaction; the affected account's balance reflects the change automatically since it is computed at read time (see FR-2.2), not stored or separately updated.	Must
FR-3.3	The system shall allow filtering of transactions by date range, category, account, and type.	Must
FR-3.4	The system shall provide a fixed set of 8–10 default expense/income categories, plus a basic form for a user to add their own category (name and type). Full category management — editing, deleting, or reassigning	Must

ID	Requirement	Priority
	transactions on category removal — is out of scope for this version.	

3.4 Module: Analytics & Reporting

ID	Requirement	Priority
FR-4.1	The system shall display a dashboard summarizing total income, total expense, and net savings for a selected period.	Must
FR-4.2	The system shall render a category-wise expense breakdown as a chart.	Must
FR-4.3	The system shall render an income-vs-expense trend chart over a selectable date range (e.g., last 6 months).	Must
FR-4.4	The system shall allow export of transaction data and reports as CSV. Attempt only after all Must and Should items across every module are complete.	Could

4. Non-Functional Requirements

4.1 Performance

- Standard CRUD API responses shall complete within 500ms under normal load (single-user demo/testing conditions).
- Dashboard analytics queries shall be optimized using MongoDB aggregation pipelines rather than client-side computation over raw data.

4.2 Security

- Passwords shall be hashed (bcrypt) before storage; plaintext passwords shall never be persisted or logged.
- All API endpoints shall require a valid JWT, except registration, login, and the password-reset request/confirm endpoints, which validate a single-use, time-limited reset token instead (see FR-1.4, FR-1.7).
- Input validation and sanitization shall be enforced on both client and server to prevent injection attacks.
- Sensitive configuration (JWT secret, database URI) shall be stored in environment variables, never committed to source control.

4.3 Usability

- The interface shall be responsive across desktop, tablet, and mobile viewport widths, achieved via a component library's built-in responsive grid (e.g., Tailwind CSS or MUI) rather than hand-tuned custom breakpoints.
- Forms shall provide inline validation feedback (e.g., invalid amount, missing required field) before submission.
- Core actions (add transaction, view balance) shall be reachable within two clicks from the dashboard.

4.4 Reliability

- The system shall maintain data consistency between account balances and their underlying transactions at all times (no orphaned or unreflected transactions).

4.5 Scalability

- The database schema shall support growth in transaction volume per user without requiring structural redesign (indexed queries on user ID and date).

4.6 Maintainability

- The codebase shall separate concerns across models, controllers, routes, and services in the Express backend.
- Core business logic (balance aggregation, category-wise reporting) shall be covered by unit tests (Jest) to allow safe refactoring.

4.7 Availability

- A formal uptime percentage (e.g., 95%) is not meaningful for a single EC2 instance with no monitoring infrastructure in place, and is not claimed. Instead: the deployed application shall be manually verified as accessible and fully functional immediately before each demo or evaluation session.

5. External Interface Requirements

5.1 User Interfaces

- Login / Registration screens
- Dashboard (summary cards, charts)
- Accounts view (list, create, edit, delete)
- Transactions view (list, filters, add/edit form)

- Reports view (charts, CSV export — stretch goal)

5.2 Software Interfaces

- Frontend (React) communicates with backend exclusively via a REST API under the /api/v1 prefix (JSON over HTTPS).
- Backend (Express/Node) communicates with MongoDB via Mongoose ODM.

5.3 Hardware Interfaces

- None. FinManage is a standard web application with no direct hardware dependency.

5.4 Communication Interfaces

- All client-server communication shall occur over HTTPS in the deployed environment.

6. Database Requirements

FinManage uses MongoDB as its primary data store, with Mongoose schemas enforcing structure. The core collections and their key fields are described below.

6.1 Collection: Users

Field	Type	Notes
_id	ObjectId	Primary key
name	String	Required
email	String	Required, unique, indexed
passwordHash	String	Required; bcrypt hash, never returned in API responses
resetPasswordToken	String	Hashed reset token; null when no reset is pending
resetPasswordExpires	Date	Expiry timestamp for the reset token (supports FR-1.7)
createdAt	Date	Auto-generated

6.2 Collection: Accounts

Field	Type	Notes
_id	ObjectId	Primary key
userId	ObjectId	References Users; indexed
name	String	e.g., 'HDFC Savings'
type	String (enum)	cash | bank | credit_card | savings
openingBalance	Number	Required

6.3 Collection: Transactions

Field	Type	Notes
_id	ObjectId	Primary key
userId	ObjectId	References Users; indexed
accountId	ObjectId	References Accounts; indexed
type	String (enum)	income | expense
amount	Number	Required, positive
categoryId	ObjectId	References Categories; indexed
date	Date	Required; indexed for range queries
notes	String	Optional

6.4 Collection: Categories

Field	Type	Notes
_id	ObjectId	Primary key
userId	ObjectId	Null for system default categories
name	String	e.g., 'Groceries', 'Rent'
type	String (enum)	income | expense

6.5 Key Relationships

- One User has many Accounts and Transactions (1:N).
- One Account has many Transactions (1:N); an Account cannot be hard-deleted while Transactions reference it.
- One Category has many Transactions via categoryId (1:N).

7. System Architecture Overview

FinManage follows a standard three-tier MERN architecture:

- Presentation Tier: React SPA, consuming REST endpoints via Axios, using React Router for navigation and Context/useState for state management.
- Application Tier: Node.js + Express REST API, organized into routes, controllers, and service modules; JWT middleware guards protected routes.
- Data Tier: MongoDB, accessed via Mongoose schemas/models, hosted on MongoDB Atlas.

Deployment: the frontend is built as static assets served either via a CDN or the Express server; the backend runs in a Docker container deployed to an AWS EC2 instance via manual build and deploy steps. Automated CI/CD (e.g., Jenkins) is not required for this version.

8. Requirement Prioritization Summary (MoSCoW)

This summary consolidates priorities across all functional modules to guide build order.

Priority	Meaning	Approx. Count
Must	Core to a working, presentable system; non-negotiable for demo	16
Should	No additional requirements in this version	0
Could	Nice-to-have; only if time remains after all Must items are complete	1
Won't (this version)	Explicitly out of scope — see Section 1.2	—

Recommended build order: complete all Must-have requirements across Authentication and Accounts first (Sections 3.1–3.2), then core Transaction Management (3.3), followed by Analytics (3.4). Password reset (FR-1.7) is Must-have per course requirement but is functionally isolated from the rest, so it can be built independently without blocking anything else. This sequencing keeps a functional, demonstrable product available at every stage of development.

— End of Document —